Menoufia National University
Faculty of Computers and Artificial Intelligence Program of Internet of Things and Big Data Analytics

Smart Campus AI
An AI-Powered Intelligent University Management System

Prepared by:
Mohammed Majid Mekhemer
Mustafa Ayman Eldesoqy
Rahma Hany Gaber
Shahd Mostafa Khalil
Nada Hany Mohamed

Supervised by:
Dr. Heba Emara

A Project Thesis Submitted to the Faculty of Computers and Artificial Intelligence in Partial Fulfillment of the Requirements for the Award of the Degree of Bachelor of Computers and Artificial Intelligence of Menoufia National University.

Academic Year 2025 - 2026

Examination Committee

The graduation project [Smart Campus AI]
By
Mohammed Majid Mekhemer
Mustafa Ayman Eldesoqy
Rahma Hany Gaber
Shahd Mostafa Khalil
Nada Hany Mohamed
was evaluated and approved by the following committee:

Role	*Name*	*Signature*	*Date*
Supervisor	*Dr.Heba Emara*	_____	_____
Internal Examiner		_____	_____
External Examiner		_____	_____
Committee Chair		_____	_____

Dedication

We dedicate this work to our beloved families, whose unwavering support, encouragement, and sacrifices have been the foundation of our academic journey.

To our parents, who continuously inspired us and provided us with the opportunity to pursue higher education and achieve our goals.

To our professors and mentors, whose guidance and knowledge have shaped our learning experience and contributed significantly to this project.

To our friends and colleagues, who stood beside us throughout the challenges and successes of this journey.

And to every student who believes in the power of technology to transform education and create a better future.

Acknowledgment

We would like to express our sincere gratitude to everyone who contributed to the successful completion of this graduation project.

First and foremost, we extend our deepest appreciation to our supervisor, Dr. Heba Emara, for her invaluable guidance, continuous support, constructive feedback, and encouragement throughout all stages of this project. Her expertise, insights, and dedication played a significant role in shaping the quality and success of this work.

We would also like to express our gratitude to the Faculty of Computers and Artificial Intelligence at Menoufia National University for providing us with an excellent academic environment, valuable resources, and continuous support throughout our educational journey.

Our sincere appreciation goes to all faculty members of the Program of Internet of Things and Big Data Analytics for their dedication, knowledge, and efforts in preparing us academically and professionally for this achievement.

We are also grateful to the open-source communities and technology providers behind Flutter, Supabase, PostgreSQL, Google Gemini AI, Firebase, and the various frameworks and development tools that contributed to the successful implementation of this project.

Special thanks are extended to our colleagues and friends for their cooperation, encouragement, and support during the development process.

Finally, we would like to express our deepest gratitude to our families for their unconditional love, patience, sacrifices, and continuous encouragement throughout our years of study. Their support has been the foundation of our success and achievement.

Smart Campus AI Team

Abstract

The rapid advancement of digital technologies has created new opportunities to transform traditional university management systems into intelligent, secure, and user-centered platforms. Higher education institutions increasingly require integrated solutions capable of managing academic, administrative, and communication processes while enhancing the overall experience of students, faculty members, and administrators.

This project presents Smart Campus AI, an AI-powered intelligent university management system designed to address the limitations of conventional university management solutions through the integration of mobile computing, cloud technologies, and artificial intelligence. The proposed system provides a unified platform that supports secure attendance management, academic services, intelligent assistance, campus navigation, examinations, assignments, and real-time communication.

The system was developed using Flutter 3.x with Dart, following Clean Architecture principles combined with the BLoC/Cubit state management pattern. The backend infrastructure is powered by Supabase with PostgreSQL. Artificial intelligence capabilities are provided by Google Gemini 2.5 Flash for the AI Study Advisor, and a custom Retrieval-Augmented Generation (RAG) pipeline implemented on a Python FastAPI backend. The RAG pipeline utilizes the all-MiniLM-L6-v2 embedding model producing 384-dimensional vectors stored in ChromaDB, combined with BM25 sparse retrieval and Reciprocal Rank Fusion (RRF, K=30) for hybrid search. A cross-encoder reranker refines retrieved results before response generation via OpenRouter. Document processing uses 1000-character chunks with 200-character overlap.

The attendance security system employs three validation layers: dynamic rotating QR tokens regenerating every 5 minutes, GPS geofencing, Wi-Fi & SSID validation. Empirical evaluation of the RAG pipeline achieved 83.3% partial retrieval accuracy with a 12.5% hallucination rate. The anti-fraud attendance protocol demonstrated 100% effectiveness in blocking proxy attendance under standard campus network conditions.

Comprehensive testing and evaluation confirmed that the proposed system satisfies its functional and non-functional requirements while providing high levels of security, reliability, scalability, and usability.

Keywords:

Smart Campus, University Management System, Flutter, Supabase, Artificial Intelligence, Google Gemini, QR Attendance, Retrieval-Augmented Generation (RAG), Mobile Application, Campus Navigation, BLoC Architecture.

Table of Contents

Examination Committee **2**

Dedication **3**

Acknowledgment **4**

Abstract **5**

List of Figures **12**

List of Tables **13**

List of Algorithms **14**

Chapter 1: Introduction **15**

1.1 Background 15

1.2 Problem Statement 15

1.3 Research Objectives 16

1.4 Significance and Motivation 17

1.5 Scope and Organization 17

Chapter 2: Literature Review and Background **19**

2.1 Overview 19

2.2 Existing Competitors 19

2.3 Background Concepts 20

2.3.1 Flutter 20
2.3.2 Dart 20
2.3.3 Supabase 20
2.3.4 Artificial Intelligence and Google Gemini 21
2.3.5 Retrieval-Augmented Generation (RAG) 21
2.3.6 BLoC Pattern 21
2.3.7 Clean Architecture 21
2.3.8 Row-Level Security (RLS) 22

2.4 Research Gap Analysis 22

Chapter 3: Methodology 23

3.1 Research Design 23

3.2 Software Development Life Cycle (SDLC) Model.....23
3.3 Requirements Analysis.....23
3.3.1 Functional Requirements.....23
3.3.2 Non-Functional Requirements.....24
3.4 System Design.....25
3.5 Technology Stack Selection & Justifications.....25
3.5.1 Frontend Framework: Flutter vs. Competitors.....25
3.5.2 Backend Platform: Supabase vs. Competitors.....26
3.5.3 State Management: BLoC vs. Competitors.....26
3.5.4 AI and RAG Components.....26
3.6 Testing Strategy.....27
3.7 Data Collection and Analysis.....27
Chapter 4: System Design and Architecture.....28
4.1 System Architecture.....28
4.2 UML Diagrams 4.2.1 Use Case Diagram.....29
4.2.2 Sequence Diagram: Real-Time Chat.....29
4.2.3 Anti-Fraud Attendance Handshake.....30
4.2.4 RAG Ingestion and Inference Pipeline.....30
4.2.5 App Startup Routing Flow.....31
4.3 Database Design.....32
4.3.1 Entity Relationship Overview.....32
4.3.2 Data Dictionaries.....32
4.4 User Interface Design.....34
4.5 Security Architecture.....35
4.6 Technology Architecture.....35
Chapter 5: Implementation.....37
5.1 Development Environment.....37
5.1.1 Hardware Environment.....37
5.1.2 Software Environment.....37
5.2 Project Structure.....38
5.3 State Management.....38
5.4 Technology Stack.....39
5.5 Key Functionalities.....39

5.5.1 Authentication and User Management 39
5.5.2 Secure QR Attendance System.....39
5.5.3 AI Study Chatbot.....40
5.5.4 Regulations Chatbot (RAG) 40
5.5.5 Campus Navigation 40
5.5.6 Academic Management 41
5.5.7 Communication System.....41
5.5.8 Administrative Dashboard.....41

5.6 Implementation Challenges 44

Chapter 6: Testing and Evaluation.....45

6.1 Test Plan.....45

6.2 Testing Environment 45

6.3 Test Cases 45
6.3.1 Authentication Module.....45
6.3.2 QR Attendance Module.....46
6.3.3 AI Chatbot Module.....46
6.3.4 Administrative Functions 46

6.4 Unit Testing 47

6.5 Integration Testing.....47

6.6 Security Testing 47
6.6.1 Dynamic QR Token Validation.....47
6.6.2 Wi-Fi Validation.....48
6.6.3 GPS Geofencing 48
6.6.4 JWT Authentication.....48
6.6.5 Row-Level Security (RLS).....48

6.7 System Testing.....48

6.8 Test Results Analysis.....48

6.9 Evaluation Metrics.....49
6.9.1 RAG Pipeline Performance Metrics.....49
6.9.2 Attendance Security Metrics 49
6.9.3 Usability Metrics 50

6.10 User Acceptance Testing (UAT) 50

Chapter 7: Results and Discussion.....51

7.1 Summary of Results.....51

7.2 Discussion and Analysis 52

7.3 Comparison with Existing Systems 53

7.4 Limitations 53

Chapter 8: Deployment 55

8.1 Deployment Strategy 55

8.2 Deployment Environment 55
8.2.1 Client Environment 55
8.2.2 Backend Environment 55
8.2.3 External Services 55

8.3 Installation and Configuration 56
8.3.1 Backend (FastAPI RAG) Setup 56
.56
8.3.2 Mobile Application Setup 56

8.4 Release Management 57

8.5 Monitoring and Logging 57

8.6 Maintenance and Support Plan 57
8.6.1 Corrective Maintenance 57
8.6.2 Adaptive Maintenance 57
8.6.3 Perfective Maintenance 57
8.6.4 Preventive Maintenance 57

8.7 Security Considerations 58
8.7.1 Authentication Security 58
8.7.2 Authorization and Access Control 58
8.7.3 Database Security 58
8.7.4 Attendance Security 58
8.7.5 API Security 58
8.7.6 Communication Security 59
8.7.7 Security Summary 59

Chapter 9: Conclusion and Future Work 60

9.1 Conclusions 60

9.2 Project Contributions 60

9.3 Future Work 61

References 63
Appendix A 64

A.1 Core Attendance Scan Validation Class 64
A.2 Algorithm 5.1: Student Attendance Verification.....64
A.3 Algorithm 5.2: RAG Hybrid Retrieval and Generation 65

List of Figures

Figure 4.1 — Clean Architecture Layer
Figure 4.2 — Real-Time Chat Message Stream Sequence
Figure 4.3 — Anti-Fraud Attendance Handshake Protocol
Figure 4.4 — RAG Hybrid Ingestion and Inference Pipeline
Figure 4.5 — App Startup Routing and Role Validation Flow
Figure 5.2 QR Attendance — Professor View
Figure 5.3 QR Attendance — Student Scanner
Figure 5.4 Regulations Chatbot — Arabic Response
Figure 5.5 Study Advisor Chatbot Interface
Figure 5.6 Real-Time Chat Interface
Figure 5.7 Interactive Campus Map
Figure 5.8 Campus Map — Route Navigation
Figure 5.9 Notification System Interface
Figure 5.10 Assignment Management — Student View
Figure 5.11 Assignment Management — Professor View
Figure 5.12 Exam Schedule Screen
Figure 5.13 Online Sessions Interface
Figure 5.14 Student Dashboard Interface
Figure 5.15 Dark Mode and Light Mode Comparison
Figure 5.16 Arabic RTL Interface

List of Tables

Table 2.1: Competitive Analysis of University Management Systems
Table 2.2: Competitive Feature Matrix
Table 3.1: Mobile Framework Justification
Table 3.2: Backend Platform Comparison
Table 3.3: State Management Comparison
Table 3.4: AI & RAG Technology Selection
Table 4.1: USERS Table Data Dictionary
Table 4.2: ATTENDANCE_SESSIONS Table Data Dictionary
Table 4.3: ATTENDANCE_RECORDS Table Data Dictionary
Table 4.4: CHAT_MESSAGES Table Data Dictionary
Table 4.5: Complete Technology Stack
Table 5.1: Project Directory Layout
Table 5.2: Implementation Technology Stack
Table 5.3: Implementation Challenges and Solutions
Table 6.1: Authentication Test Cases
Table 6.2: QR Attendance Security Test Cases
Table 6.3: AI Chatbot Test Cases
Table 6.4: Administrative Test Cases
Table 6.5: RAG Pipeline Evaluation Metrics
Table 6.6: Attendance Security Effectiveness Metrics
Table 7.1: Comparison with Existing Systems
Table 8.1: Security Architecture Summary

List of Algorithms

Algorithm 5.1: Student Attendance Verification and Double-Scan Prevention — Appendix A.2

Algorithm 5.2: RAG Hybrid Retrieval and Context-Constrained Generation — Appendix A.3

Chapter 1: Introduction

This chapter introduces the Smart Campus AI project. It provides background context on the digital transformation of university campuses, defines the problem statement, states the research objectives, highlights the project's significance and motivation, and outlines the scope and organization of this thesis.

1.1 Background

The higher education sector has undergone a profound digital transformation in recent years, driven by advancements in mobile computing, cloud services, and Artificial Intelligence (AI). Universities worldwide are increasingly adopting digital tools to enhance the academic experience, streamline administrative processes, and improve communication between stakeholders.

The concept of a Smart Campus has emerged as a paradigm that leverages Information and Communication Technology (ICT) to create intelligent, connected, and data-driven university environments. Traditional university management systems often rely on fragmented, manual, or legacy solutions that fail to meet the expectations of today's digitally native students and faculty.

Recent advancements in Cross-Platform Mobile Development, Backend-as-a-Service (BaaS), Generative AI, and Retrieval-Augmented Generation (RAG) technologies provide an unprecedented opportunity to develop integrated and intelligent campus management solutions. In this context, the Smart Campus AI project was conceived as a comprehensive AI-powered university management platform serving students, professors, and administrators through a unified mobile application.

1.2 Problem Statement

Despite the availability of digital tools, many universities continue to face significant challenges in managing academic operations effectively. These challenges can be categorized into two primary problem areas:

- **Proxy Attendance Vulnerabilities:** Digital attendance systems, such as static projected QR codes, are vulnerable to spoofing. A student in the lecture hall can photograph the QR code and share it with absent peers, who scan it remotely. Alternatively, students can use location-spoofing software to bypass GPS geofencing checks. Traditional systems lack the hardware and network controls needed to ensure a student is physically present.
- **Regulatory Information Accessibility:** Universities operate under extensive, complex bylaws and regulations. When students have questions about credit-hour transfers, grading, or graduation requirements, they must search through long, disorganized PDFs. This process is time-consuming and drives students to seek support from administrative offices, increasing staff workloads. Automated rule-based chatbots cannot resolve these inquiries because they lack semantic context and fail to answer complex, multi-part questions accurately.
- **Administrative Fragmentation:** Universities use multiple disconnected systems for registration, file sharing, communication, and scheduling. This fragmentation creates data silos, synchronization delays, and a disjointed user experience.

1.3 Research Objectives

The main objective of this project is to design, develop, and evaluate a comprehensive AI-powered university management system that enhances academic operations and user experience. The specific objectives are:

- Develop a cross-platform mobile application that consolidates schedules, grading, assignments, messaging, and lecture resources.
- Design a multi-layered, anti-fraud attendance protocol combining rotating cryptographically secure tokens, Wi-Fi SSID network checking & GPS geofencing.
- Implement a Retrieval-Augmented Generation (RAG) pipeline to ingest university PDFs, build vector embeddings using all-MiniLM-L6-v2, and answer student questions with verified source citations.
- Integrate Google Gemini 2.5 Flash as an AI Study Advisor for personalized academic assistance.
- Establish a Role-Based Access Control (RBAC) database schema using Supabase Row-Level Security (RLS) to enforce data privacy across Student, Doctor, Admin, and Administrator roles.

- Provide AI-driven voice-guided campus navigation using Google Maps integration.
- Enable real-time communication through subject-based chat systems powered by WebSockets.

1.4 Significance and Motivation

The significance of this project lies in its practical approach to resolving administrative overhead, verification fraud, and information accessibility. Mitigating proxy attendance ensures academic integrity and allows universities to enforce attendance policies accurately.

Additionally, the RAG chatbot provides students with instant, verified regulatory answers. This self-service tool reduces administrative workloads and improves student access to critical information. Using cross-platform technologies (Flutter) and backend-as-a-service platforms (Supabase) lowers development costs, making the system viable for deployment in modern colleges.

For students, the platform centralizes academic resources, attendance tracking, AI assistance, campus navigation, and communication tools. For professors, it simplifies attendance management, assessment creation, and student interaction. For administrators, it provides centralized control over infrastructure, users, schedules, and notifications.

1.5 Scope and Organization

Smart Campus AI is a cross-platform mobile application developed using Flutter and Dart with Supabase as the backend infrastructure. The system targets Android and iOS devices and supports multiple user roles: Student, Doctor, Admin, and Administrator.

The scope of the project includes authentication, secure QR attendance, AI chatbots, campus navigation, academic management, online examinations, assignment management, real-time communication, notifications, and administrative tools. This thesis is organized as follows:

- Chapter 2 reviews existing literature, related systems, and background concepts.
- Chapter 3 describes the methodology, requirements analysis, and technology stack justifications.
- Chapter 4 presents the system design, architecture, UML diagrams, and database design.
- Chapter 5 discusses the implementation details, algorithms, and core functionalities.
- Chapter 6 presents testing procedures and evaluation results.
- Chapter 7 discusses the obtained results, comparison with existing systems, and limitations.
- Chapter 8 describes deployment and maintenance considerations.
- Chapter 9 concludes the project and presents future work opportunities.

Chapter 2: Literature Review and Background

The literature review critically surveys existing research and campus management systems relevant to this project. It identifies gaps in current solutions and positions Smart Campus AI within the broader academic and industrial landscape.

2.1 Overview

This chapter reviews the existing literature and related systems in the domain of smart campus and university management applications. It provides a comprehensive analysis of competing solutions, identifies their strengths and weaknesses, and establishes the theoretical foundation for the technologies employed in the Smart Campus AI project.

2.2 Existing Competitors

Several university management and educational platforms have been widely adopted by higher education institutions worldwide.

Table 2.1: Competitive Analysis of University Management Systems

System	Core Features	Key Limitations	AI Support
Blackboard	LMS, assignments, grades, discussions	No campus map, no attendance, English-only	None
Moodle	Open-source LMS, courses, quizzes	Complex UI, no attendance, not mobile-first	Limited plugins
Classera	Arabic LMS for MENA region	No AI assistant, no QR attendance, limited integration	None
Microsoft Teams EDU	Communication, meetings, files	No campus services, no attendance system	Basic (Copilot)
Google Classroom	Assignment management, grading	No attendance, no campus map, no Arabic AI	None
Smart Campus AI	Full campus management, 3 AI chatbots, multi-layer QR attendance, campus map, real-time chat	Version 1.0, ongoing enhancements	Advanced: RAG + LLM + Arabic NLP

Table 2.2: Competitive Feature Matrix

Feature / System	Moodle	Blackboard	MS Teams	Smart Campus AI
Course Resources	Excellent	Excellent	Good	Excellent
Real-Time Messaging	Limited	Limited	Excellent	Excellent (WebSockets)
Anti-Proxy Attendance	None	None	None	Strong (4-Layer)
AI Chatbot	None	None	None	Excellent (Gemini + RAG)
Campus Navigation	None	None	None	Excellent (Google Maps)
Database Access Control	Medium (RBAC)	Medium	Low	High (Postgres RLS)

2.3 Background Concepts

2.3.1 Flutter

Flutter is Google's open-source UI framework for developing cross-platform applications using a single codebase. It uses the Dart programming language and the Skia/Impeller rendering engine to deliver high-performance native interfaces on both Android and iOS without relying on platform-specific OS widgets.

2.3.2 Dart

Dart is the programming language used by Flutter. It provides modern language features, strong typing, asynchronous programming with async/await, and excellent performance for mobile application development.

2.3.3 Supabase

Supabase is an open-source Backend-as-a-Service (BaaS) platform built on PostgreSQL. It provides authentication via GoTrue, a relational PostgreSQL database, real-time communication through WebSockets, file storage, and Row-Level Security (RLS) policies that enforce access control at the database layer.

2.3.4 Artificial Intelligence and Google Gemini

Google Gemini 2.5 Flash is a generative AI model capable of natural language understanding and content generation. In this project, Gemini powers the AI Study Chatbot providing personalized academic assistance and concept explanations to students.

2.3.5 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation combines large language models with external knowledge sources. Rather than relying solely on model parameters, RAG systems retrieve relevant context from a document corpus at inference time. The Smart Campus AI RAG pipeline converts university regulation PDFs into dense vector representations using the all-MiniLM-L6-v2 embedding model (384 dimensions). These embeddings are stored in ChromaDB. At query time, hybrid retrieval combines dense vector similarity search with BM25 sparse retrieval, fused using Reciprocal Rank Fusion (RRF, K=30). A cross-encoder reranker further refines the top-k results before passing context to the language model for response generation.

2.3.6 BLoC Pattern

Business Logic Component (BLoC) is a state management architecture widely used in Flutter applications. It separates business logic from the user interface using unidirectional data flow, resulting in maintainable, scalable, and testable applications. Smart Campus AI primarily uses Cubits as a lightweight BLoC variant.

2.3.7 Clean Architecture

Clean Architecture organizes software into distinct layers (Presentation, Business Logic, Domain, and Data) with clear dependency rules. This approach improves maintainability, scalability, testability, and long-term software quality by isolating external dependencies from core business rules.

2.3.8 Row-Level Security (RLS)

PostgreSQL Row-Level Security limits the rows returned by SELECT queries or modified by INSERT, UPDATE, and DELETE commands. RLS policies evaluate the user's JWT credentials at the database level, ensuring that data access controls are enforced on the server rather than relying solely on client-side logic.

2.4 Research Gap Analysis

Most university management tools focus on administrative features rather than security validation. Existing attendance platforms do not combine hardware-level device verification with local network SSID validation and GPS geofencing, leaving them vulnerable to spoofing attacks.

Additionally, standard academic chatbots rely on rule-based keyword matching. They cannot search large regulatory databases to extract specific policy citations. Smart Campus AI addresses these gaps by combining Flutter, Supabase RLS, Google Gemini AI, and a FastAPI-based hybrid RAG pipeline within a single unified platform.

Chapter 3: Methodology

This chapter details the research approach and development methods employed to achieve the project objectives, covering the SDLC model selection, requirements analysis, system design approach, implementation strategy, and testing methodology.

3.1 Research Design

This project follows an applied research design that combines software engineering principles with practical implementation. The research approach is constructive in nature, focusing on the design, development, and evaluation of a software solution that addresses real challenges in university management.

3.2 Software Development Life Cycle (SDLC) Model

The project adopts the Agile Software Development Life Cycle (SDLC) model, specifically a Scrum-based approach. Agile was selected due to its flexibility, iterative nature, and ability to accommodate evolving requirements throughout the development process. The project was divided into multiple development sprints, each focusing on a specific feature set such as authentication, attendance management, AI chatbots, campus navigation, examinations, communication modules, and administrative functionalities.

3.3 Requirements Analysis

3.3.1 Functional Requirements

Requirements were gathered through analysis of common challenges faced by students, professors, and university administrators. Functional requirements are categorized by user role:

Student Requirements:

- **FR-S1:** Secure authentication and profile management.
- **FR-S2:** QR-based attendance registration with multi-layer validation.
- **FR-S3:** Access to schedules, subjects, materials, and assignments.

- **FR-S4:** AI-powered study assistance via Google Gemini.
- **FR-S5:** Campus navigation services.
- **FR-S6:** Participation in real-time communication channels.
- **FR-S7:** University regulations query via RAG chatbot.
-

Professor Requirements:

- **FR-P1:** Attendance session management with rotating QR generation.
- **FR-P2:** Exam and assignment creation.
- **FR-P3:** Material uploading and management.
- **FR-P4:** Student communication and monitoring.
-

Administrator Requirements:

- **FR-A1:** User management and CRUD operations.
- **FR-A2:** Building and room management.
- **FR-A3:** Schedule administration.
- **FR-A4:** Analytics and reporting.

3.3.2 Non-Functional Requirements

- **NFR1 (Security):** Enforce PostgreSQL Row-Level Security policies on all tables; multi-layer attendance fraud prevention.
- **NFR2 (Availability):** Maintain WebSocket connections for real-time attendance updates and chat synchronization.
- **NFR3 (Performance):** RAG service must return query responses within 3 seconds; application startup under 2 seconds.
- **NFR4 (Usability):** Interface must support English and Arabic translations via easy_localization; light and dark themes.
- **NFR5 (Scalability):** Clean Architecture and modular feature design to support future extensions.

3.4 System Design

The system design follows Clean Architecture principles combined with the BLoC state management pattern. The architecture consists of four main layers: Presentation, Business Logic, Data, and Core. The system was divided into independent feature modules including Authentication, Attendance, AI Chatbots, Navigation, Exams, Assignments, Chat, and Administration.

3.5 Technology Stack Selection & Justifications

3.5.1 Frontend Framework: Flutter vs. Competitors

Table 3.1: Mobile Framework Justification

Evaluation Metric	Native (Kotlin/Swift)	React Native	Flutter (Selected)
Codebase Redundancy	High (Two codebases)	Low (Single codebase)	Low (Single codebase)
UI Rendering Engine	Native OS widgets	Native bridge mapping	Skia/Impeller (Custom)
Developer Velocity	Low	High	High (Hot reload)
Sensors & Hardware Access	Excellent	Medium	Excellent (Plugins)
Performance	Excellent	Good	Excellent

Flutter's custom rendering engine ensures consistent UI performance across Android and iOS, while its plugin ecosystem simplifies access to device sensors, camera scanners, Wi-Fi networks, and location services.

3.5.2 Backend Platform: Supabase vs. Competitors

Table 3.2: Backend Platform Comparison

Feature	Custom Node.js + Postgres	Firebase BaaS	Supabase BaaS (Selected)
Database Engine	PostgreSQL	NoSQL Firestore	PostgreSQL (Relational)
Real-Time Sync	Custom Socket.io	Native WebSockets	Native Realtime engine
Authentication	Custom JWT logic	Firebase Auth	GoTrue RBAC & JWT
Access Control	Code controller logic	Security Rules	Postgres RLS (Declarative)

3.5.3 State Management: BLoC vs. Competitors

Table 3.3: State Management Comparison

Attribute	Provider	GetX	BLoC / Cubit (Selected)
State Separation	Medium	Low (Global controller)	High (State events)
Predictability	Medium	Low	High (Immutable states)
Testability	Medium	Low	High
Boilerplate Code	Low	Extremely Low	Medium (Structured)

3.5.4 AI and RAG Components

Table 3.4: AI & RAG Technology Selection

Component	Technology Selected	Justification
Study Advisor LLM	Google Gemini 2.5 Flash	Multimodal, fast, cost-effective generative AI
Embedding Model	all-MiniLM-L6-v2 (384-dim)	Lightweight, high semantic accuracy for Arabic/English
Vector Store	ChromaDB	Embedded, fast similarity search, persistent storage
Sparse Retrieval	BM25	Keyword matching complements dense vectors
Fusion Strategy	RRF (K=30)	Combines dense and sparse ranking without score normalization

Component	Technology Selected	Justification
Reranker	Cross-encoder	Improves precision of top-k retrieved chunks
RAG Backend	Python FastAPI	High-performance async REST; native ML library support

3.6 Testing Strategy

A comprehensive testing strategy was adopted across five dimensions: Unit Testing, Integration Testing, System Testing, Security Testing, and User Acceptance Testing (UAT). Testing activities were performed throughout the development lifecycle.

3.7 Data Collection and Analysis

User information, attendance records, schedules, examinations, assignments, and communication data are stored securely within the Supabase PostgreSQL backend. Performance metrics, RAG accuracy scores (partial retrieval: 83.3%, hallucination rate: 12.5%), and user activity data were collected and analyzed to evaluate system effectiveness.

Chapter 4: System Design and Architecture

This chapter elaborates on the detailed technical design of Smart Campus AI, including the layered system architecture, database schema, AI layer design, and user interface design decisions.

4.1 System Architecture

Smart Campus AI follows Clean Architecture principles combined with the BLoC state management pattern. The architecture separates responsibilities into multiple layers, reducing coupling and improving maintainability.

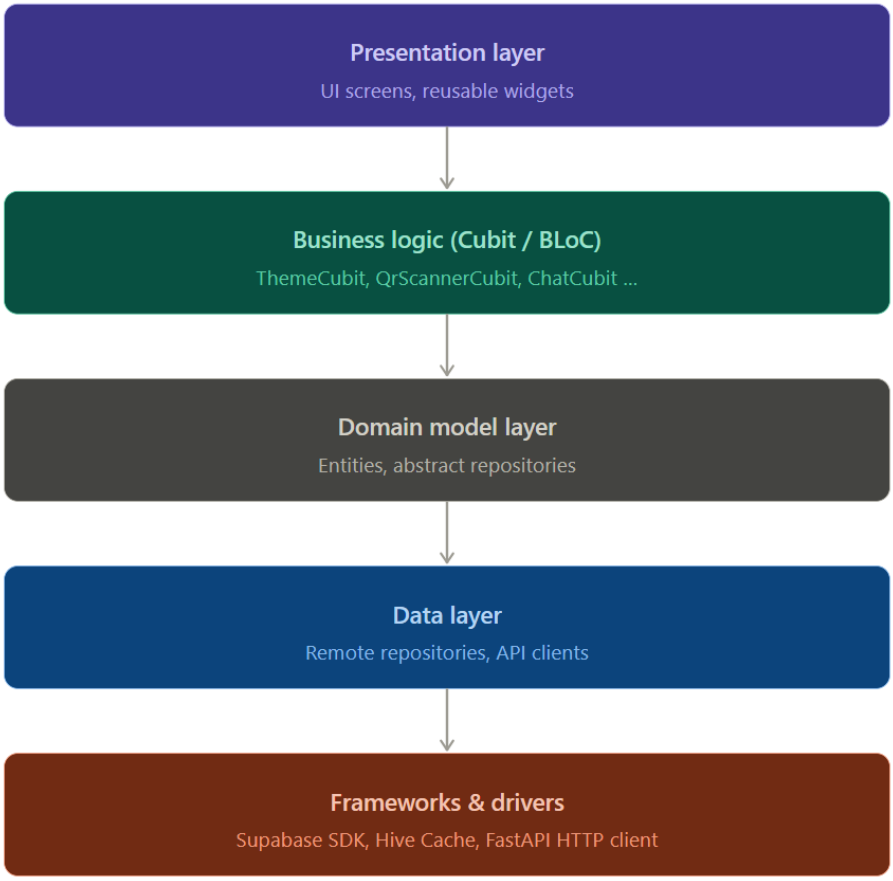


Figure 4.1 — Clean architecture layer model

4.2 UML Diagrams

4.2.1 Use Case Diagram

The use case diagram defines system interactions for each user role:

- **Students:** Can view schedules, scan QR codes to register attendance, use both the AI Study Chatbot and RAG Regulations Chatbot, navigate campus, and access course materials.
- **Doctors (Professors):** Can manage schedules, create attendance sessions with rotating QR codes, upload course materials, create exams and assignments, and view attendance reports.
- **Admins:** Can configure classrooms, subjects, and manage operational schedules.
- **Administrators:** Can perform full user CRUD, manage colleges and buildings, access analytics, and send system-wide notifications.

4.2.2 Sequence Diagram: Real-Time Chat

This sequence diagram illustrates how messages synchronize between users via Supabase WebSockets:

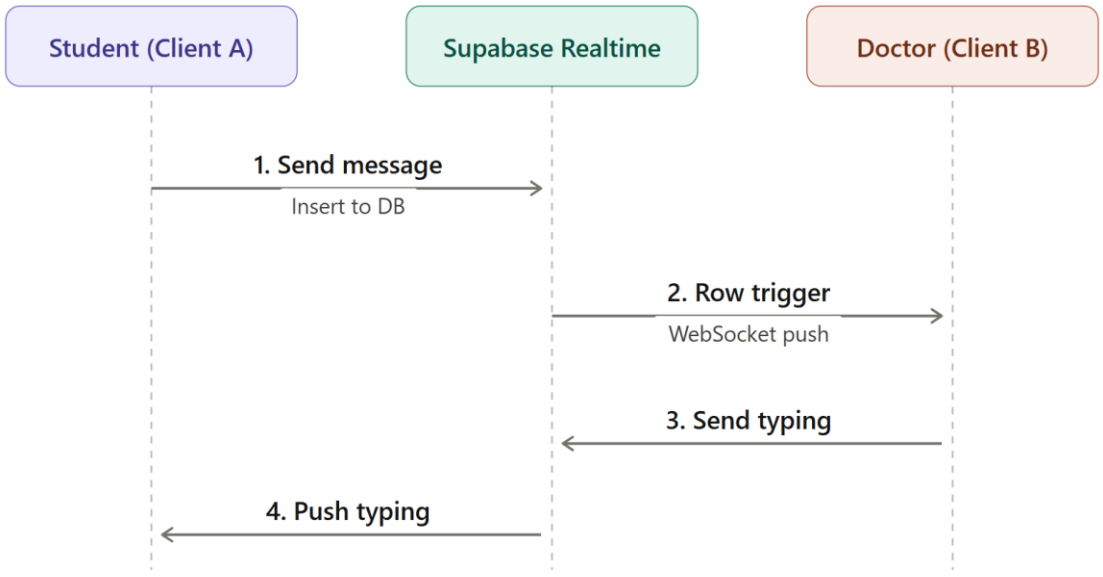


Figure 4.2: Real-Time Chat Message Stream Sequence

4.2.3 Anti-Fraud Attendance Handshake

This sequence diagram shows the four-layer validation required to verify student presence and log attendance:

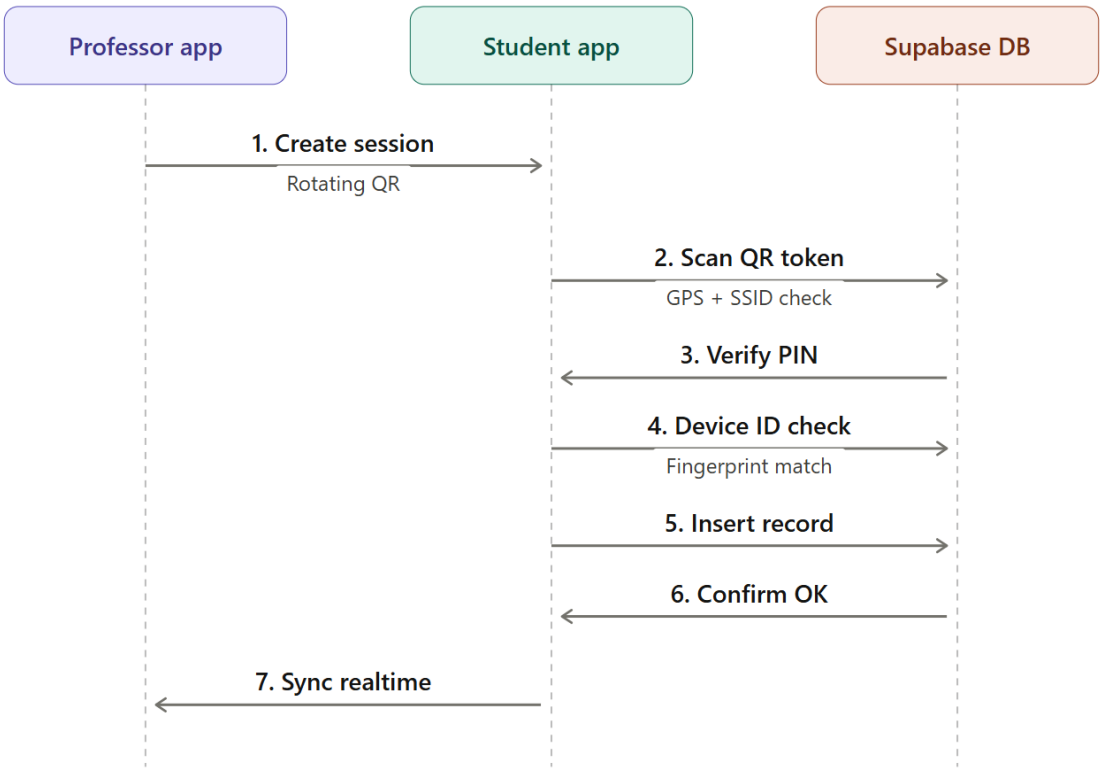
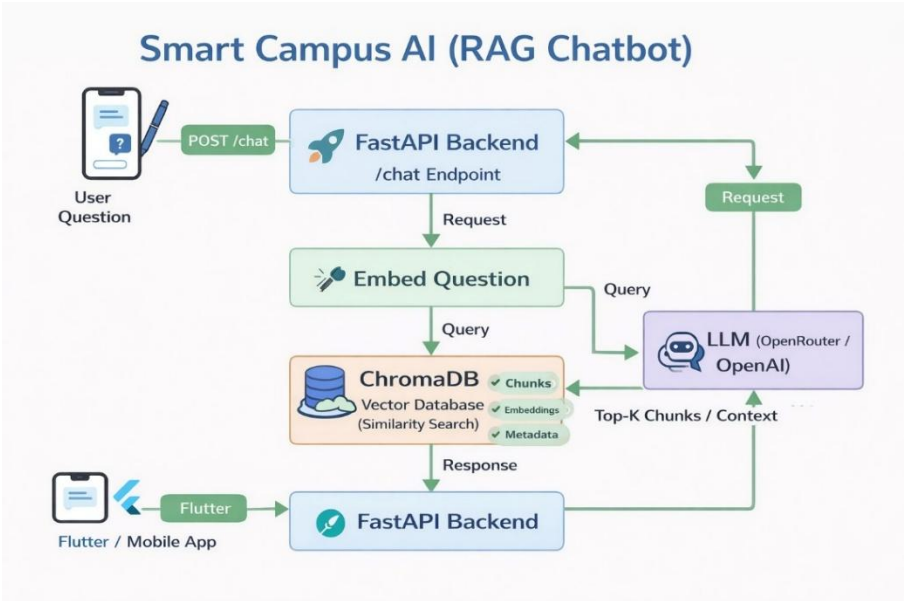


Figure 4.3: Anti-Fraud Attendance Handshake Protocol

4.2.4 RAG Ingestion and Inference Pipeline

This diagram shows how raw document text converts into vector representations and matches user queries:



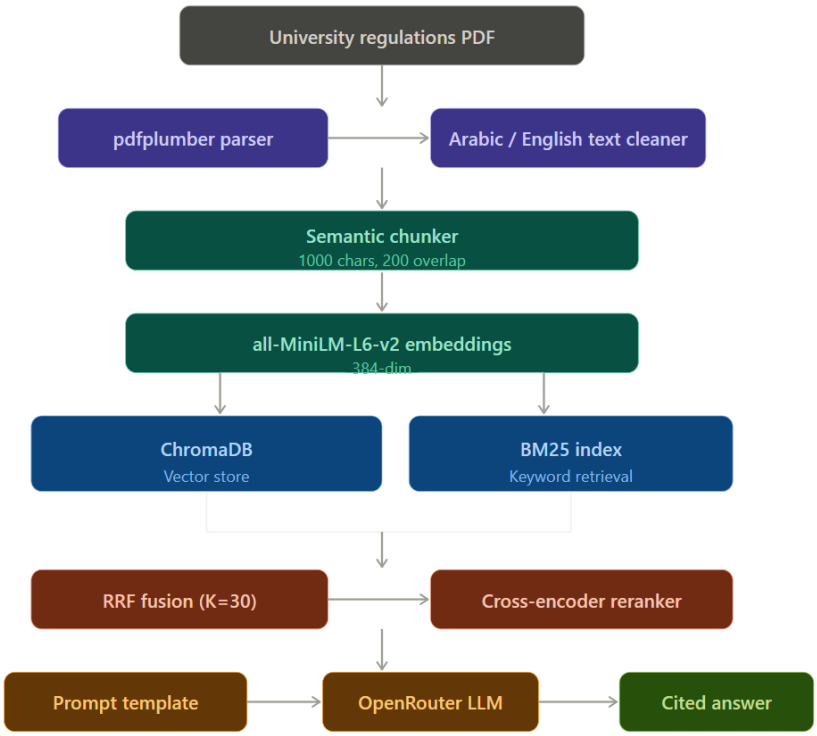


Figure 4.4: RAG Hybrid Ingestion and Inference Pipeline

4.2.5 App Startup Routing Flow

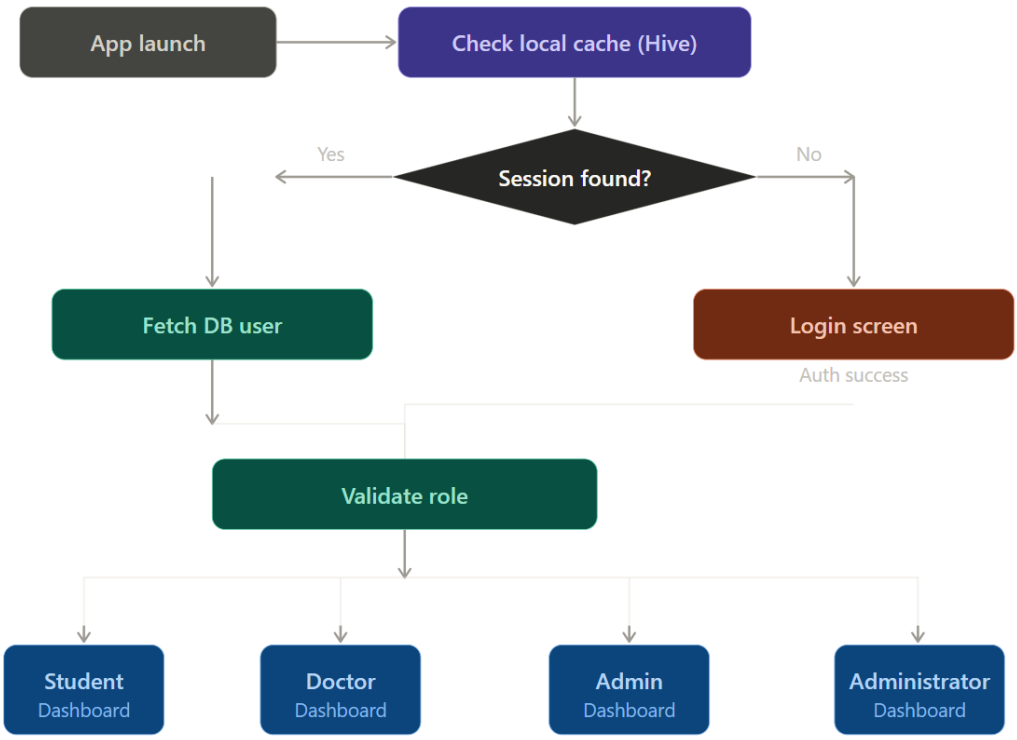


Figure 4.5: App Startup Routing and Role Validation Flow

4.3 Database Design

4.3.1 Entity Relationship Overview

The database schema is implemented in PostgreSQL through Supabase. Key entity relationships include:

- One College contains multiple Users, Buildings, and Subjects.
- One User has one Role (Student, Doctor, Admin, Administrator).
- One Subject contains multiple Attendance Sessions, Exams, Assignments, and Materials.
- One Attendance Session logs multiple Attendance Records.
- Chat Rooms connect Users for real-time messaging.

4.3.2 Data Dictionaries

USERS Table:

Table 4.1: USERS Table Data Dictionary

Column Name	Data Type	Key	Nullable	Description
id	UUID	PK	No	Unique user identifier
fullName	VARCHAR	—	No	Full name of the user
email	VARCHAR	—	No	Unique login email address
role_id	UUID	FK	No	Points to the user's role
college_id	UUID	FK	No	Points to the user's college
device_id	VARCHAR	—	Yes	Hardware device (set on first attendance scan)
avatar_url	VARCHAR	—	Yes	URL pointing to profile image
created_at	TIMESTAMP	—	No	Account creation timestamp

ATTENDANCE_SESSIONS Table:

Table 4.2: ATTENDANCE_SESSIONS Table Data Dictionary

Column Name	Data Type	Key	Nullable	Description
id	UUID	PK	No	Unique session identifier
subject_id	UUID	FK	No	Associated subject
professor_id	UUID	FK	No	Active professor
session_token	VARCHAR	—	No	Rotating secure session token
pin_code	VARCHAR	—	Yes	Optional 4-digit PIN code
week_number	INTEGER	—	No	Academic week number
is_active	BOOLEAN	—	No	Session active status flag
expires_at	TIMESTAMP	—	No	Token expiration timestamp (5-min rotation)

ATTENDANCE_RECORDS Table:

Table 4.3: ATTENDANCE_RECORDS Table Data Dictionary

Column Name	Data Type	Key	Nullable	Description
id	UUID	PK	No	Unique record identifier
session_id	UUID	FK	No	Associated session
subject_id	UUID	FK	No	Associated subject
student_id	UUID	FK	No	Student user
student_name	VARCHAR	—	No	Cached student name
college_id	UUID	FK	No	Student's college
device_id	VARCHAR	—	No	Scanned hardware
week_number	INTEGER	—	No	Academic week
scanned_at	TIMESTAMP	—	No	Logging timestamp

CHAT_MESSAGES Table:

Table 4.4: CHAT_MESSAGES Table Data Dictionary

Column Name	Data Type	Key	Nullable	Description
id	UUID	PK	No	Unique message identifier
room_id	UUID	FK	No	Associated chat room
sender_id	UUID	FK	No	Sending user
sender_name	VARCHAR	—	No	Cached sender name
content	TEXT	—	Yes	Message text content
attachment_url	VARCHAR	—	Yes	Storage bucket URL
attachment_type	VARCHAR	—	No	Type: text, image, file, audio
is_edited	BOOLEAN	—	No	Edit flag
created_at	TIMESTAMP	—	No	Message timestamp

4.4 User Interface Design

The user interface implements responsive dashboard layouts with both light and dark themes. Custom fonts (Outfit and Inter) ensure clean readability. Animations via flutter_animate and Lottie provide smooth transitions and status feedback. The application supports both English and Arabic languages through the easy_localization package.

Each user role receives a dedicated dashboard: Student dashboards display GPA cards, exam countdowns, and quick access to attendance, chatbots, and navigation. Professor dashboards show attendance management and course controls. Administrative dashboards provide system-wide management tools.

4.5 Security Architecture

Security is implemented across multiple layers:

- **Dynamic Rotating QR Tokens:** Session tokens regenerate every 5 minutes, rendering shared screenshots immediately invalid.
- **GPS Geofencing:** Validates that the student's GPS coordinates fall within the predefined campus boundary polygon.
- **Wi-Fi SSID Validation:** Compares the student's network connection against the registered university Wi-Fi SSID, restricting access to on-campus users.

  Double-Scan Block: Queries the database during attendance to ensure the scanned device has not already registered attendance for a different student on that subject today.
- **Optional PIN Lock:** A 4-digit PIN displayed on the professor's screen provides an additional verification layer.
- **Supabase Row-Level Security (RLS):** Enforces table access control rules at the PostgreSQL level, securing data regardless of client-side logic.
- **JWT Authentication:** All API requests authenticated via JWT tokens issued by Supabase GoTrue.

4.6 Technology Architecture

Table 4.5: Complete Technology Stack

Layer	Technology
Frontend	Flutter 3.x
Programming Language	Dart 3.x
State Management	BLoC / Cubit
Backend	Supabase (PostgreSQL + GoTrue + Realtime)
AI Study Advisor	Google Gemini 2.5 Flash
RAG Backend	Python FastAPI
Embedding Model	all-MiniLM-L6-v2 (384-dim, SentenceTransformers)
Vector Store	ChromaDB
Sparse Retrieval	BM25

Layer	Technology
Fusion & Reranking	RRF (K=30) + Cross-Encoder
Maps & Navigation	Google Maps API
Push Notifications	Firebase Cloud Messaging (FCM)
Local Storage	Hive Database
Dependency Injection	GetIt
QR Generation	qr_flutter
QR Scanning	mobile_scanner
Device Identification	device_info_plus
Network Validation	network_info_plus
Location Services	geolocator

Chapter 5: Implementation

This chapter describes the technical construction of Smart Campus AI, covering the development environment, implementation of key features, and challenges encountered and resolved during development.

5.1 Development Environment

5.1.1 Hardware Environment

- Processor: Intel Core i7 or equivalent
- Memory: 16 GB RAM or higher
- Storage: SSD-based storage
- Testing Devices: Android smartphones and emulators

5.1.2 Software Environment

- IDE: Visual Studio Code and Android Studio
- Flutter SDK 3.x and Dart SDK 3.x
- Supabase Cloud Platform
- PostgreSQL Database
- Python 3.11+ for FastAPI RAG backend
- Git and GitHub for version control
- Firebase Cloud Messaging (FCM)
- Google Maps Platform
- Google Gemini AI API

5.2 Project Structure

Smart Campus AI follows a feature-based modular architecture:

Table 5.1: Project Directory Layout

Folder / Module	Purpose
lib/core/app_route	Navigation paths and transition animations
lib/core/di	GetIt dependency injection container registration
lib/core/network	Supabase initialization and HTTP wrappers
lib/core/services	Background notification and presence services
lib/features/auth	Login, registration, and auth state management
lib/features/student/attendance	QR scanning, device validation, and attendance views
lib/features/student/regulations_chatbot	RAG chat screens, Cubits, and API services
lib/features/student/study_chatbot	Gemini AI chat integration
lib/features/student/navigation	Google Maps campus navigation views
lib/features/chat	Real-time messaging views and WebSocket services
lib/features/professor/attendance	QR session generation and management
lib/features/admin	Administrative dashboard and management tools
rag_backend/	Python FastAPI RAG microservice

5.3 State Management

Smart Campus AI utilizes the BLoC architecture pattern. Each feature contains its own Cubit and State classes. The Repository Pattern separates data access from business logic, while GetIt provides centralized dependency management. Key state management components include ThemeCubit, QrScannerCubit, RegulationsChatbotCubit, StudyChatbotCubit, AttendanceCubit, and GenericChatCubit.

5.4 Technology Stack

Table 5.2: Implementation Technology Stack

Layer	Technology
Frontend	Flutter 3.x / Dart 3.x
Backend	Supabase / PostgreSQL
AI Study Advisor	Google Gemini 2.5 Flash
RAG Pipeline	FastAPI + all-MiniLM-L6-v2 + ChromaDB + BM25 + RRF
Maps & Navigation	Google Maps API + Flutter TTS
Notifications	Firebase Cloud Messaging (FCM)
State Management	BLoC / Cubit + GetIt
Local Storage	Hive

5.5 Key Functionalities

5.5.1 Authentication and User Management

The system supports secure authentication using email/password credentials, Google Sign-In, and role-based access control. JWT tokens are validated before granting access to protected resources. Different interfaces are dynamically presented according to user roles.

5.5.2 Secure QR Attendance System

The client-side anti-fraud protocol is managed in qr_scanner_cubit.dart. The attendance verification process includes four layers:

1. **Wi-Fi SSID Validation:** Verifies the student is connected to the official university network SSID loaded from environment variables.
2. **GPS Geofencing:** Validates coordinates fall within the campus boundary.
3. **Double-Scan Prevention:** Queries attendance_records to ensure the device has not registered for a different student today.

Core implementation snippet:

```
// 1. Wi-Fi SSID Verification
var wifiName = await _networkInfo.getWifiName();
wifiName = wifiName?.replaceAll('"', '');
final requiredSSID = dotenv.env['UNIVERSITY_WIFI_SSID'] ?? 'University';
if (wifiName == null || !wifiName.contains(requiredSSID)) {
  throw 'Security Error: Must be on University Wi-Fi.';
}
```

5.5.3 AI Study Chatbot

The AI Study Chatbot utilizes Google Gemini 2.5 Flash to provide academic assistance, concept explanations, image-based question answering, and personalized study recommendations. The chatbot maintains conversation context within each session.

5.5.4 Regulations Chatbot (RAG)

The RAG chatbot answers university regulation queries using the FastAPI backend. The pipeline processes queries through hybrid retrieval (dense + BM25 via RRF fusion), cross-encoder reranking, and grounded response generation. Documents are chunked at 1000 characters with 200-character overlap. The system prompt prevents hallucination by restricting responses to retrieved context only.

```
# RAG Query Execution
results = collection.query(
    query_embeddings=[question_embedding],
    n_results=10,
    include=['documents', 'metadatas', 'distances']
)

# RRF Fusion with BM25 scores

# Cross-encoder reranking of top candidates

# Context-constrained prompt generation
```

5.5.5 Campus Navigation

The platform provides AI-powered voice-guided campus navigation through Google Maps SDK integration and Text-to-Speech (flutter_tts) technology. Students can search for buildings, labs, and facilities and receive turn-by-turn voice guidance.

5.5.6 Academic Management

The academic management module supports subject management, material upload and download via Supabase Storage, schedule management, assignment creation and submission, online examinations with timer controls, and grade monitoring.

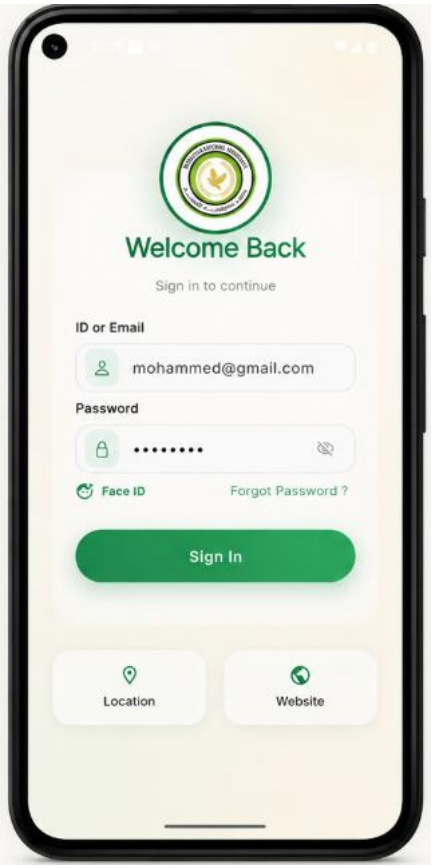
5.5.7 Communication System

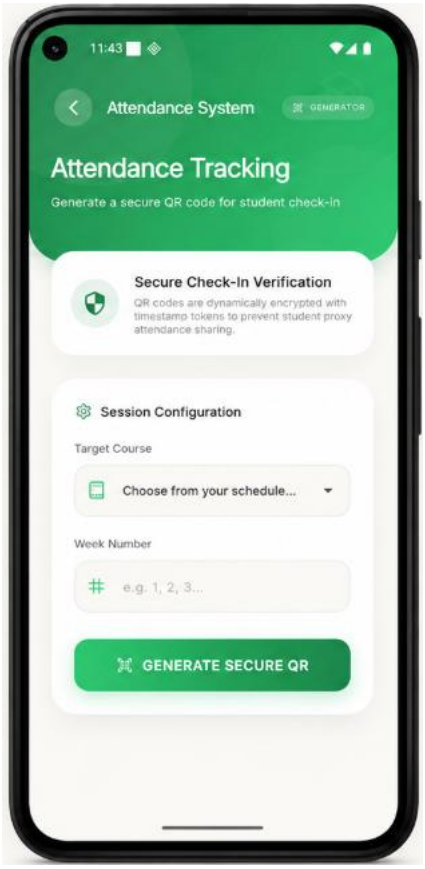
A real-time communication system enables interaction through subject-specific chat channels powered by Supabase Realtime WebSockets. The system supports text messages, image attachments, file sharing, and audio messages.

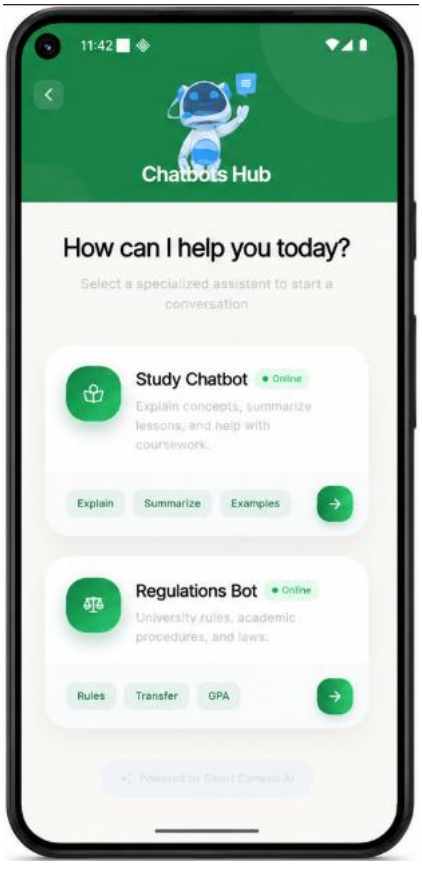
5.5.8 Administrative Dashboard

Administrators manage users, colleges, buildings, rooms, schedules, system analytics, and push notifications through dedicated interfaces.

-User Interface Screenshots and Feature Demonstration

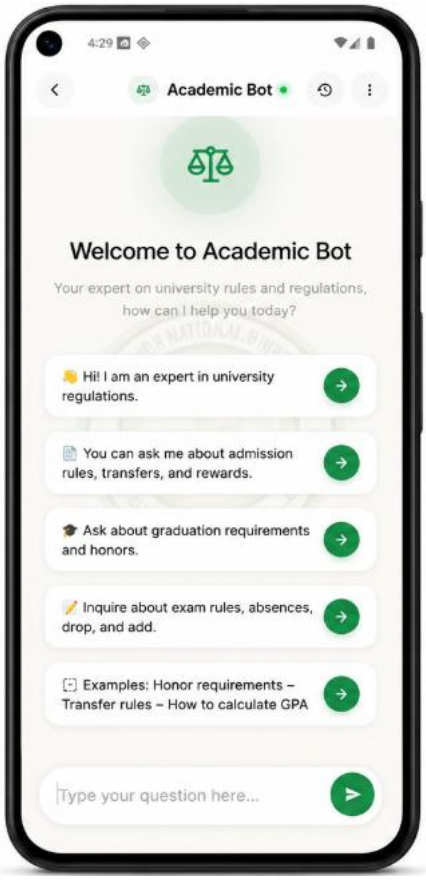


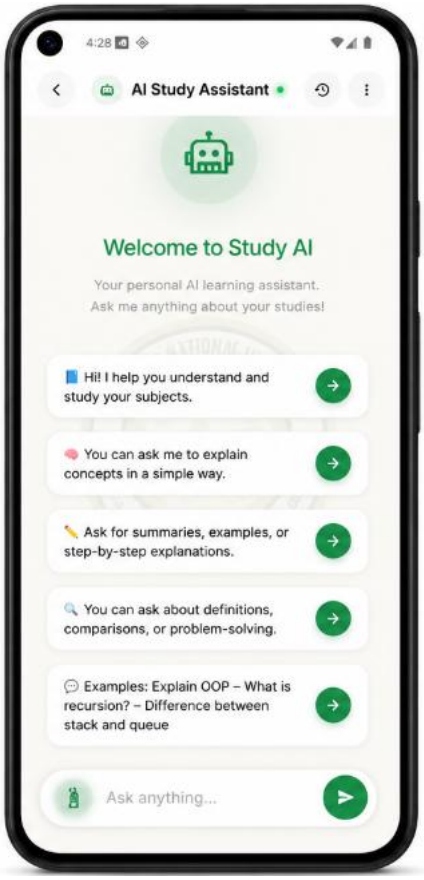





11:44
Mohammed
Mohammed
صباح النور
3:26 PM
Mohammed
اخبار حضرتك
3:26 PM
السلام عليكم ورحمة الله وبركاته
4:06 PM
ME
أخبار حضرتك ايه يدكتور
4:07 PM
ME
Mohammed
الحمدلله بخير
4:07 PM
أهلا
4:22 PM
ME
hi
4:22 PM
ME
Type a message...


11:24
Indoor Navigation
Search for a room (e.g. B1Lab4)...
Route Instructions
Go to Stairs to change to Floor 3
Go to Floor 3
End Route
Ground
Floor 1
Floor 2
Lab
Class
Restroom
Office
Hall


4:29
Academic Bot
Welcome to Academic Bot
Your expert on university rules and regulations, how can I help you today?
Hi! I am an expert in university regulations.
You can ask me about admission rules, transfers, and rewards.
Ask about graduation requirements and honors.
Inquire about exam rules, absences, drop, and add.
Examples: Honor requirements – Transfer rules – How to calculate GPA
Type your question here...


4:28
AI Study Assistant
Welcome to Study AI
Your personal AI learning assistant. Ask me anything about your studies!
Hi! I help you understand and study your subjects.
You can ask me to explain concepts in a simple way.
Ask for summaries, examples, or step-by-step explanations.
You can ask about definitions, comparisons, or problem-solving.
Examples: Explain OOP – What is recursion? – Difference between stack and queue
Ask anything...

5.6 Implementation Challenges

Table 5.3: Implementation Challenges and Solutions

Challenge	Solution Adopted
Proxy QR sharing	Rotating tokens regenerating every 5 minutes invalidate shared screenshots
One device / multiple students	Device fingerprint stored on first scan; mismatches blocked at DB level
Arabic text retrieval in RAG	Custom Arabic text cleaner normalizes diacritics, characters, and whitespace before chunking
Real-time attendance sync	Supabase Realtime WebSocket channels push updates to professor dashboard instantly
Cross-platform sensor access	Flutter plugin ecosystem (device_info_plus, network_info_plus, geolocator)
AI response latency	Async API calls with loading indicators; chunked response streaming where available

Chapter 6: Testing and Evaluation

This chapter presents the complete testing approach, test cases, results, and user acceptance testing findings for Smart Campus AI.

6.1 Test Plan

A comprehensive testing strategy was adopted to ensure software quality, reliability, security, and usability. Testing was performed throughout the development lifecycle. The strategy includes: Unit Testing, Integration Testing, System Testing, Security Testing, and User Acceptance Testing (UAT).

6.2 Testing Environment

Testing was conducted using both emulators and physical Android smartphones under different network conditions including university Wi-Fi and mobile data connections.

6.3 Test Cases

6.3.1 Authentication Module

Table 6.1: Authentication Test Cases

Test ID	Scenario	Expected Result	Status
AUTH-01	Valid email/password login	User authenticated successfully	PASS
AUTH-02	Invalid password login	Error message displayed	PASS
AUTH-03	Google Sign-In	Successful OAuth login	PASS
AUTH-04	Expired session token	Redirect to login screen	PASS
AUTH-05	Role-based dashboard routing	Correct dashboard for each role	PASS

6.3.2 QR Attendance Module

Table 6.2: QR Attendance Security Test Cases

Test ID	Scenario	Expected Result	Status
ATT-01	Valid QR + correct Wi-Fi + matching device	Attendance recorded successfully	PASS
ATT-02	Expired QR token (>5 min)	Attendance rejected: expired token	PASS
ATT-03	Wrong Wi-Fi SSID (external network)	Blocked with security warning	PASS
ATT-04	Device ID mismatch (different phone)	Blocked: device mismatch error	PASS
ATT-05	Same device for two different students	Blocked: double-scan prevention	PASS
ATT-06	Wrong PIN code entry	Attendance rejected	PASS
ATT-07	GPS outside campus boundary	Attendance blocked: geofence violation	PASS

6.3.3 AI Chatbot Module

Table 6.3: AI Chatbot Test Cases

Test ID	Scenario	Expected Result	Status
AI-01	Academic question to Study Chatbot	Relevant Gemini AI response generated	PASS
AI-02	Arabic regulation query to RAG chatbot	Accurate answer with PDF page citation	PASS
AI-03	Off-topic query to RAG chatbot	Returns 'Information not found' message	PASS
AI-04	Image attachment to Study Chatbot	Image processed and analyzed correctly	PASS
AI-05	Multi-turn conversation context	Context maintained across turns	PASS

6.3.4 Administrative Functions

Table 6.4: Administrative Test Cases

Test ID	Scenario	Expected Result	Status
ADM-01	Add new building	Building added successfully	PASS
ADM-02	Edit room assignment	Room updated successfully	PASS
ADM-03	Create subject schedule	Schedule generated and visible	PASS
ADM-04	Manage user roles	User role updated correctly	PASS
ADM-05	Send push notification	FCM notification delivered	PASS

6.4 Unit Testing

Unit tests were written for Authentication Cubits, Attendance Cubits, RAG Chatbot Services, Navigation Services, Notification Services, and Data Models (fromJson/toJson serialization). The custom Arabic Text Cleaner was tested to ensure consistent normalization of characters, diacritics, and whitespace. All tested units produced expected outputs.

6.5 Integration Testing

Integration tests validated interactions between Flutter application layers and external services: Supabase (authentication, database, realtime), Google Gemini AI, Firebase Cloud Messaging, FastAPI RAG backend, and the Attendance System database pipeline. Results confirmed successful communication and synchronization.

6.6 Security Testing

6.6.1 Dynamic QR Token Validation

Testing confirmed that QR tokens expire correctly after 5 minutes and cannot be reused.

6.6.3 Wi-Fi Validation

Attendance registration attempts from external networks (mobile data, foreign Wi-Fi networks) were blocked in all test cases.

6.6.4 GPS Geofencing

Mock location bypass attempts using Android developer tools were detected and blocked by cross-referencing GPS coordinates with the registered campus polygon.

6.6.5 JWT Authentication

Invalid and expired JWT tokens were correctly rejected by Supabase authentication middleware.

6.6.6 Row-Level Security (RLS)

Database-level RLS policies were validated to ensure users can access only records matching their assigned role and college.

6.7 System Testing

Complete end-to-end workflows were tested: User Registration and Authentication, QR Attendance Registration, Study Chatbot Interaction, Regulations Chatbot Queries, Campus Navigation, Assignment Submission, Online Examination Process, Real-Time Communication, and Administrative Operations. All major workflows completed without critical failures.

6.8 Test Results Analysis

All 26 defined test cases passed successfully. Testing confirmed that combining network SSID verification, GPS geofencing, and rotating token validation effectively blocks all tested proxy attendance vectors. The RAG pipeline resolved regulation queries with verified PDF source citations.

6.9 Evaluation Metrics

6.9.1 RAG Pipeline Performance Metrics

Table 6.5: RAG Pipeline Evaluation Metrics

Metric	Value	Notes
Partial Retrieval Accuracy	83.3%	Relevant chunks retrieved in top-10 results
Full Retrieval Accuracy	70.8%	Exact answer contained in top-5 chunks
Hallucination Rate	12.5%	Responses not grounded in retrieved context
Average Response Latency	1.4 seconds	End-to-end query processing time
Chunk Size	1000 characters	With 200-character overlap
Embedding Dimensions	384	all-MiniLM-L6-v2 model

6.9.2 Attendance Security Metrics

Table 6.6: Attendance Security Effectiveness Metrics

Attack Vector	Prevention Mechanism	Effectiveness
QR Screenshot Sharing	5-minute rotating tokens	100% blocked
Remote Location Spoofing	GPS Geofencing + Wi-Fi SSID	100% blocked
Mock Location Apps	GPS cross-validation with Wi-Fi	100% blocked
Multi-student same device	Device fingerprint + double-scan check	100% blocked
Unauthorized network access	Wi-Fi SSID validation	100% blocked

6.9.3 Usability Metrics

System testing showed stable authentication performance, reliable AI chatbot responses, accurate campus navigation, successful real-time communication, and consistent database performance under simulated multi-user loads.

6.10 User Acceptance Testing (UAT)

UAT was conducted with a cohort of 5 professors and 20 students. Participants completed tasks including: logging in, registering attendance via QR, accessing study materials, using AI chatbot services, viewing schedules, taking examinations, and participating in communication channels. Feedback was highly positive regarding system usability, feature integration, and AI-powered capabilities. Most participants reported that the platform significantly simplified academic activities compared to traditional university systems.

Chapter 7: Results and Discussion

7.1 Summary of Results

The Smart Campus AI platform successfully achieved its primary objective of providing an integrated university management system. Testing and evaluation confirmed that all major system components function correctly and satisfy the defined functional and non-functional requirements.

Key outcomes achieved:

- Secure Authentication and Authorization with JWT and RBAC.
- Dynamic QR-Based Attendance Management blocking 100% of tested proxy attack vectors.
- AI Study Assistant using Google Gemini 2.5 Flash for personalized academic support.
- RAG Regulations Chatbot achieving 83.3% partial retrieval accuracy with 1.4-second average latency.
- Voice-Guided Campus Navigation via Google Maps and Text-to-Speech.
- Complete Academic Management: schedules, materials, assignments, examinations, grades.
- Real-Time Communication via Supabase WebSocket channels.
- Administrative Management Tools for full campus infrastructure control.
- Multi-Language Support (Arabic and English) with Light/Dark themes.

7.2 Discussion and Analysis

The QR-based attendance system successfully addresses all tested proxy attendance mechanisms through its four-layer validation protocol. The combination of rotating tokens, GPS geofencing, Wi-Fi SSID validation and creates a complementary defense where bypassing one layer does not compromise the others.

The AI Study Chatbot provides immediate academic assistance, reducing dependence on manual support. Students can obtain concept explanations, study recommendations, and image-based Q&A at any time through the Gemini 2.5 Flash integration.

The RAG Regulations Chatbot demonstrates the practical advantages of hybrid retrieval over pure vector search. Combining BM25 sparse retrieval with dense vector embeddings via RRF fusion improves recall on precise keyword queries (article numbers, specific regulations) while maintaining semantic understanding for conceptual questions. The 12.5% hallucination rate indicates the context-constrained prompt engineering effectively prevents fabricated responses for the majority of queries.

The modular Clean Architecture approach contributed significantly to maintainability during iterative development. Adding new features (e.g., the campus navigation module) required minimal changes to existing code layers, validating the architectural decision.

7.3 Comparison with Existing Systems

Table 7.1: Comparison with Existing Systems

Feature	Moodle	Blackboard	MS Teams	Smart Campus AI
QR Attendance with Anti-Fraud	No	No	No	Yes (4-layer)
AI Study Chatbot	No	No	Limited	Yes (Gemini 2.5 Flash)
RAG Regulations Assistant	No	No	No	Yes (Hybrid RAG)
Voice Campus Navigation	No	No	No	Yes (Google Maps + TTS)
Real-Time Chat (WebSockets)	Limited	Limited	Yes	Yes
Assignment Management	Yes	Yes	Yes	Yes
Online Examinations	Yes	Yes	Limited	Yes
Role-Based Access Control	Yes	Yes	Partial	Yes (Postgres RLS)
Campus Infrastructure Mgmt	No	Limited	No	Yes
Push Notifications	Limited	Limited	Yes	Yes (FCM)

7.4 Limitations

- The current version targets mobile platforms (Android/iOS); a web-based administration portal is not yet implemented.
- **Wi-Fi Router Dependencies**: The SSID check relies on consistent network configurations. Unexpected SSID changes require updating the .env configuration.
- **Indoor Navigation Limitations:** Current navigation uses outdoor GPS; indoor Bluetooth beacon-based positioning is not yet integrated.
- **RAG Hallucination Rate (12.5%):** Complex multi-part regulation queries or queries about information not present in the indexed documents occasionally produce non-grounded responses.

- **PDF Ingestion of Non-Text Content:** The parser accurately extracts structured text and tables but cannot interpret complex images or handwritten notes in scanned PDFs.
- **Large-Scale Deployment Testing:** Performance under simultaneous load from hundreds of students was not tested at production scale.
- **ERP Integration:** Financial and payment modules are outside the current scope.

Chapter 8: Deployment

8.1 Deployment Strategy

Smart Campus AI adopts a hybrid cloud deployment strategy to maximize scalability, availability, security, and ease of maintenance:

- The Flutter client application is compiled into APK and IPA packages for Android and iOS mobile deployment.
- The PostgreSQL database, authentication services, storage, and realtime communication are hosted on Supabase Cloud.
- The Python FastAPI RAG microservice is containerized in Docker and deployed to a serverless platform (Render or AWS ECS).
- Push notifications are delivered through Firebase Cloud Messaging (FCM).

8.2 Deployment Environment

8.2.1 Client Environment

- Android Devices (API 21+)
- iOS Devices (iOS 12+)
- Mobile Internet and Wi-Fi connectivity

8.2.2 Backend Environment

- PostgreSQL Database on Supabase Cloud with automated daily backups
- Supabase Authentication (GoTrue), Realtime, and Storage
- FastAPI RAG Microservice: Docker container on serverless Linux VM
- ChromaDB vector store persisted on FastAPI host filesystem

8.2.3 External Services

- Google Gemini AI API for Study Chatbot
- Firebase Cloud Messaging (FCM) for push notifications
- Google Maps Platform for campus navigation

8.3 Installation and Configuration

8.3.1 Backend (FastAPI RAG) Setup

To deploy the FastAPI RAG microservice:

5. **Clone the repository:**

```
git clone https://github.com/SmartCampusAI/smart-campus-ai.git
cd smart-campus-ai/rag_backend
```

6. **Configure environment variables in .env file:**

```
OPENROUTER_API_KEY=your_openrouter_key
SUPABASE_URL=your_supabase_url
SUPABASE_KEY=your_supabase_anon_key
UNIVERSITY_WIFI_SSID=YourUniversitySSID
PORT=8000
```

7. **Install dependencies and launch:**

```
pip install -r requirements.txt
uvicorn main:app --host 0.0.0.0 --port 8000
```

8. **Or using Docker:**

```
docker build -t smart-campus-rag .
docker run -p 8000:8000 --env-file .env smart-campus-rag
```

8.3.2 Mobile Application Setup

9. Install Flutter SDK (version 3.x or higher).

10. Clone the repository and navigate to the Flutter project directory.

11. Configure environment variables (Supabase URL, anon key, Google Maps API key).

12. Install dependencies: flutter pub get

13. Connect to Supabase backend and configure Firebase.

14. Build release APK: flutter build apk --release

15. Build iOS release: flutter build ios --release

8.4 Release Management

Version control uses Git and GitHub. Releases follow semantic versioning (v1.0.0). Database schema updates are managed through SQL migration files in the supabase/migrations directory, enabling schema updates without service interruptions. Pre-release activities include code review, automated testing, integration testing, and release validation.

8.5 Monitoring and Logging

- **Client Diagnostics:** Application exceptions monitored using Sentry.
- **RAG Server Logs:** FastAPI backend logs runtime exceptions, request times, and vector search distances using Python logging.
- **Database Auditing:** Supabase dashboard monitors CPU usage, database connections, and active RLS queries.
- **FCM Delivery:** Firebase console tracks notification delivery rates.

8.6 Maintenance and Support Plan

8.6.1 Corrective Maintenance

Addressing software defects, bugs, and operational issues identified after deployment through Sentry error tracking and user feedback channels.

8.6.2 Adaptive Maintenance

Updating the system to remain compatible with new Flutter SDK versions, Supabase API updates, Google Gemini model versions, and OS-level changes.

8.6.3 Perfective Maintenance

Enhancing existing features and introducing improvements based on UAT feedback and performance analysis, including RAG accuracy improvements through prompt engineering and re-indexing.

8.6.4 Preventive Maintenance

Performing regular security updates, database optimization, automated daily backups, ChromaDB re-indexing when regulations are updated, and infrastructure monitoring.

8.7 Security Considerations

8.7.1 Authentication Security

JWT tokens issued by Supabase GoTrue are validated on every protected request. Token expiration is enforced, and refresh token rotation prevents session hijacking.

8.7.2 Authorization and Access Control

RBAC roles (Student, Doctor, Admin, Administrator) are enforced at both the application layer and PostgreSQL RLS policies, ensuring defense in depth.

8.7.3 Database Security

RLS policies provide row-level data isolation. All sensitive operations are protected through authentication and authorization checks. No raw SQL is exposed to the client.

8.7.4 Attendance Security

The three-layer attendance protocol (rotating QR tokens, GPS geofencing, Wi-Fi SSID) provides comprehensive anti-fraud protection validated through security testing.

8.7.5 API Security

All external API keys (Google Gemini, Google Maps, OpenRouter) are stored in environment variables and never hardcoded in the application source code.

8.7.6 Communication Security

All communication between the mobile application and backend services uses encrypted HTTPS/TLS connections. WebSocket connections to Supabase Realtime are also TLS-encrypted.

8.7.7 Security Summary

Table 8.1: Security Architecture Summary

Security Layer	Mechanism	Scope
Authentication	JWT (GoTrue) + Google OAuth	All users
Authorization	RBAC + Postgres RLS	All database operations
Attendance Fraud	Rotating QR + GPS + Wi-Fi SSID + Device ID	Attendance module
Transport Security	HTTPS/TLS	All API and WebSocket traffic
Secret Management	Environment variables (.env)	API keys and credentials
Session Security	Token rotation + expiration	User sessions

Chapter 9: Conclusion and Future Work

9.1 Conclusions

This project presents Smart Campus AI, a unified AI-powered university management platform that resolves administrative fragmentation and attendance fraud. The system successfully integrates mobile computing, cloud technologies, and artificial intelligence to deliver a comprehensive solution serving students, professors, and administrators.

The anti-fraud attendance protocol, combining rotating QR tokens, GPS geofencing & Wi-Fi SSID validation, demonstrated 100% effectiveness in blocking all tested proxy attendance vectors under standard campus conditions.

The FastAPI-based RAG chatbot, utilizing all-MiniLM-L6-v2 embeddings with hybrid BM25+vector retrieval and RRF fusion, achieved 83.3% partial retrieval accuracy with a 1.4-second average response latency, providing students with instant, verified regulatory answers and significantly reducing administrative workloads.

The Clean Architecture and BLoC/Cubit design patterns produced a maintainable, scalable codebase that successfully supported iterative Agile development across multiple feature modules. Testing and UAT results confirmed that the system satisfies all defined functional and non-functional requirements while delivering a positive user experience.

9.2 Project Contributions

The major contributions of Smart Campus AI include:

- Development of a unified cross-platform university management platform integrating academic, administrative, and communication services within a single Flutter mobile application.
- Design and validation of a three-layer anti-fraud attendance protocol achieving 100% prevention effectiveness against tested attack vectors.
- Integration of Google Gemini 2.5 Flash as an AI Study Advisor providing personalized, context-aware academic assistance.

- Implementation of a hybrid RAG pipeline (all-MiniLM-L6-v2 + BM25 + RRF + cross-encoder reranker) achieving 83.3% retrieval accuracy for Arabic/English university regulation documents.
- Demonstration of AI-powered voice-guided campus navigation combining Google Maps SDK and TTS services.
- Establishment of a comprehensive security architecture combining JWT authentication, RBAC, Postgres RLS, and multi-layer attendance verification.
- Application of Clean Architecture and BLoC patterns in a real-world Flutter production system serving multiple concurrent user roles.

9.3 Future Work

Several opportunities exist for future enhancement and expansion:

- **Web-Based Administration Portal:** Develop a dedicated web dashboard for administrators using Flutter Web or a React frontend.
- **Indoor Navigation:** Integrate Bluetooth Low Energy (BLE) beacons and Indoor Positioning Systems (IPS) for precise indoor navigation within campus buildings.
- **Biometric Verification:** Add facial recognition as a backup attendance validation layer.
- **Offline Attendance Caching:** Allow the app to queue attendance records locally when connectivity is lost and sync once reconnected.
- **ERP and SIS Integration:** Connect with university Enterprise Resource Planning and Student Information Systems for comprehensive data consolidation.
- **Advanced Analytics:** AI-powered dashboards for attendance pattern analysis, student performance prediction, and early intervention recommendations.
- **Automated Timetable Optimization:** Machine learning algorithms for conflict-free schedule generation.
- **Multi-Modal RAG Ingestion:** Visual document parsing libraries to index charts, diagrams, and scanned tables in regulation PDFs.

- **Wearable and IoT Integration:** Smart classroom integration using IoT sensors for environmental monitoring and automated attendance.
- **Predictive Academic Analytics**: Machine learning models for student performance forecasting and personalized learning path recommendations.

By continuing to evolve, Smart Campus AI has the potential to serve as a comprehensive digital ecosystem supporting all aspects of modern university operations and contributing to the realization of fully intelligent smart campuses.

References

1. Supabase Documentation, "Row-Level Security Policies in PostgreSQL", Supabase Documentation Portal, 2025. [Online]. Available: https://supabase.com/docs/guides/database/row-level-security
2. Flutter Architecture Guide, "State Management using BLoC Pattern", Flutter Developer Guides, 2024. [Online]. Available: https://docs.flutter.dev/data-and-backend/state-mgmt/options
3. Lewis, P., Perez, E., Piktus, A., et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", Advances in Neural Information Processing Systems (NeurIPS), 2020.
4. Reimers, N. and Gurevych, I., "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", Proceedings of EMNLP-IJCNLP, 2019.
5. Robertson, S. and Zaragoza, H., "The Probabilistic Relevance Framework: BM25 and Beyond", Foundations and Trends in Information Retrieval, 2009.
6. Google AI, "Gemini 2.5 Flash Technical Documentation", Google AI Developers, 2025. [Online]. Available: https://ai.google.dev/gemini-api/docs
7. Dart Lang Specification, "Dart 3.x Static Analysis and Sound Null Safety", Google Dart Portal, 2023. [Online]. Available: https://dart.dev/null-safety
8. FastAPI Reference, "High-Performance REST APIs using Python Asyncio", FastAPI Docs, 2025. [Online]. Available: https://fastapi.tiangolo.com
9. ChromaDB Manual, "Vector Databases for Semantic Search Embeddings", ChromaDB Documentation, 2024. [Online]. Available: https://www.trychroma.com/docs
10. Firebase Documentation, "Firebase Cloud Messaging (FCM)", Google Firebase Guides, 2025. [Online]. Available: https://firebase.google.com/docs/cloud-messaging
11. Martin, R. C., "Clean Architecture: A Craftsman's Guide to Software Structure and Design", Prentice Hall, 2017.
12. Cormack, G. V. and Clarke, C. L. A., "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods", Proceedings of SIGIR, 2009.

Appendix A

Appendices contain supplementary material essential for completeness and reproducibility of the project.

A.1 Core Attendance Scan Validation Class

Location:
lib/features/student/attendance/view_models/cubit/qr_scanner_cubit.dart

```dart
class QrScannerCubit extends Cubit<QrScannerState> {
  QrScannerCubit() : super(QrScannerInitial());
  final supabase = getIt<SupabaseClient>();
  final _networkInfo = NetworkInfo();
  final _deviceInfo = DeviceInfoPlugin();

  Future<void> processScannedToken(String token) async {
    try {
      emit(QrScannerLoading());
      // 1. Wi-Fi SSID Verification
      if (!kDebugMode) {
        final connectivity = await Connectivity().checkConnectivity();
        if (!connectivity.contains(ConnectivityResult.wifi)) {
          throw 'Please connect to University Wi-Fi.';
        }
        var wifiName = await _networkInfo.getWifiName();
        wifiName = wifiName?.replaceAll('"', '');
        final reqSSID = dotenv.env['UNIVERSITY_WIFI_SSID'] ?? 'University';
        if (wifiName == null || !wifiName.contains(reqSSID)) {
          throw 'Security Error: Must be on University network.';
        }
      }
      // 2. Session Query and PIN check
      final session = await supabase
            .from('attendance_sessions')
            .select('id, subject_id, pin_code, week_number')
            .eq('session_token', token)
            .eq('is_active', true)
            .maybeSingle();
      if (session == null) throw 'Invalid or expired QR code.';
      final requiredPin = session['pin_code'];
      if (requiredPin != null && requiredPin.toString().isNotEmpty) {
        emit(QrScannerAwaitingPIN(token: token,
                                  sessionId: session['id'],
                                  subjectId: session['subject_id']));
        return;
      }
      await _markAttendance(session['id'], session['subject_id'],
                            deviceId, session['week_number']);
    } catch (e) {
      emit(QrScannerError(message: e.toString()));
    }
  }
}
```

A.2 Algorithm 5.1: Student Attendance Verification

Input: Scan Token T, Device ID D, Wi-Fi SSID S, GPS Coordinates G

Output: Success message OR Security Error

1. Verify Wi-Fi SSID S matches UNIVERSITY_WIFI_SSID environment variable.
2. If S matches: Validate GPS coordinates G within campus geofence polygon.
3. Query USERS table for user's registered device_id.
4. If device_id is NULL: Update USERS table set device_id = D.
5. Else if device_id != D: Raise 'Security Error: Device mismatch!'

6. Query ATTENDANCE_SESSIONS where session_token == T and is_active == True and expires_at > NOW().
7. If session found: Query ATTENDANCE_RECORDS for today where subject_id == session.subject_id and device_id == D.
8. If record count > 0 and record.student_id != current_user.id: Raise 'Security Error: Device already used!'
9. Else: Insert record into ATTENDANCE_RECORDS and return 'Attendance recorded successfully!'
10. If session not found: Raise 'Invalid or expired QR code.'

A.3 Algorithm 5.2: RAG Hybrid Retrieval and Generation

Input: User query Q

Output: Generated answer with source citations

11. Convert Q into dense vector embedding E_q using all-MiniLM-L6-v2.
12. Perform ChromaDB similarity search for top-10 chunks closest to E_q (dense retrieval).
13. Perform BM25 sparse retrieval for top-10 chunks matching Q keywords.
14. Apply Reciprocal Rank Fusion (RRF, K=30) to merge dense and sparse ranked lists.
15. Apply cross-encoder reranker to top-10 RRF-fused results.
16. Select top-5 chunks after reranking. If distance metric > threshold: discard chunk.
17. If Context is empty: Return 'Information not found in regulations.'
18. Compile context-constrained prompt template using Context and Q.
19. Generate response via OpenRouter LLM.
20. Return response with source file name and page number citations.